

6.1 Stacks

A *stack* is a collection of objects that are inserted and removed according to the *last-in, first-out (LIFO)* principle. A user may insert objects into a stack at any time, but may only access or remove the most recently inserted object that remains (at the so-called “top” of the stack). The name “stack” is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser. In this case, the fundamental operations involve the “pushing” and “popping” of plates on the stack. When we need a new plate from the dispenser, we “pop” the top plate off the stack, and when we add a plate, we “push” it down on the stack to become the new top plate. Perhaps an even more amusing example is a PEZ[®] candy dispenser, which stores mint candies in a spring-loaded container that “pops” out the topmost candy in the stack when the top of the dispenser is lifted (see Figure 6.1). Stacks are a fundamental data structure. They are used in many applications, including the following.

Example 6.1: *Internet Web browsers store the addresses of recently visited sites in a stack. Each time a user visits a new site, that site’s address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.*

Example 6.2: *Text editors usually provide an “undo” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.*

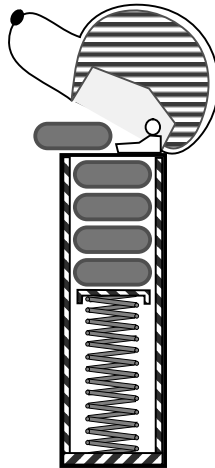


Figure 6.1: A schematic drawing of a PEZ[®] dispenser; a physical implementation of the stack ADT. (PEZ[®] is a registered trademark of PEZ Candy, Inc.)

6.1.1 The Stack Abstract Data Type

Stacks are the simplest of all data structures, yet they are also among the most important. They are used in a host of different applications, and as a tool for many more sophisticated data structures and algorithms. Formally, a stack is an abstract data type (ADT) such that an instance *S* supports the following two methods:

S.push(e): Add element *e* to the top of stack *S*.

S.pop(): Remove and return the top element from the stack *S*; an error occurs if the stack is empty.

Additionally, let us define the following accessor methods for convenience:

S.top(): Return a reference to the top element of stack *S*, without removing it; an error occurs if the stack is empty.

S.is_empty(): Return True if stack *S* does not contain any elements.

len(S): Return the number of elements in stack *S*; in Python, we implement this with the special method `--len---`.

By convention, we assume that a newly created stack is empty, and that there is no a priori bound on the capacity of the stack. Elements added to the stack can have arbitrary type.

Example 6.3: The following table shows a series of stack operations and their effects on an initially empty stack *S* of integers.

Operation	Return Value	Stack Contents
S.push(5)	—	[5]
S.push(3)	—	[5, 3]
len(S)	2	[5, 3]
S.pop()	3	[5]
S.is_empty()	False	[5]
S.pop()	5	[]
S.is_empty()	True	[]
S.pop()	“error”	[]
S.push(7)	—	[7]
S.push(9)	—	[7, 9]
S.top()	9	[7, 9]
S.push(4)	—	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop()	4	[7, 9]
S.push(6)	—	[7, 9, 6]
S.push(8)	—	[7, 9, 6, 8]
S.pop()	8	[7, 9, 6]

6.1.2 Simple Array-Based Stack Implementation

We can implement a stack quite easily by storing its elements in a Python list. The list class already supports adding an element to the end with the `append` method, and removing the last element with the `pop` method, so it is natural to align the top of the stack at the end of the list, as shown in Figure 6.2.

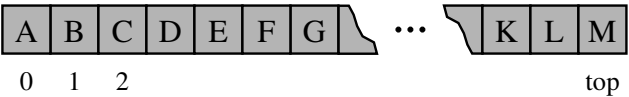


Figure 6.2: Implementing a stack with a Python list, storing the top element in the rightmost cell.

Although a programmer could directly use the list class in place of a formal stack class, lists also include behaviors (e.g., adding or removing elements from arbitrary positions) that would break the abstraction that the stack ADT represents. Also, the terminology used by the list class does not precisely align with traditional nomenclature for a stack ADT, in particular the distinction between `append` and `push`. Instead, we demonstrate how to use a list for internal storage while providing a public interface consistent with a stack.

The Adapter Pattern

The *adapter* design pattern applies to any context where we effectively want to modify an existing class so that its methods match those of a related, but different, class or interface. One general way to apply the adapter pattern is to define a new class in such a way that it contains an instance of the existing class as a hidden field, and then to implement each method of the new class using methods of this hidden instance variable. By applying the adapter pattern in this way, we have created a new class that performs some of the same functions as an existing class, but repackaged in a more convenient way. In the context of the stack ADT, we can adapt Python’s list class using the correspondences shown in Table 6.1.

<i>Stack Method</i>	<i>Realization with Python list</i>
<code>S.push(e)</code>	<code>L.append(e)</code>
<code>S.pop()</code>	<code>L.pop()</code>
<code>S.top()</code>	<code>L[-1]</code>
<code>S.is_empty()</code>	<code>len(L) == 0</code>
<code>len(S)</code>	<code>len(L)</code>

Table 6.1: Realization of a stack *S* as an adaptation of a Python list *L*.

Implementing a Stack Using a Python List

We use the adapter design pattern to define an `ArrayStack` class that uses an underlying Python list for storage. (We choose the name `ArrayStack` to emphasize that the underlying storage is inherently array based.) One question that remains is what our code should do if a user calls `pop` or `top` when the stack is empty. Our ADT suggests that an error occurs, but we must decide what type of error. When `pop` is called on an empty Python list, it formally raises an `IndexError`, as lists are index-based sequences. That choice does not seem appropriate for a stack, since there is no assumption of indices. Instead, we can define a new exception class that is more appropriate. Code Fragment 6.1 defines such an `Empty` class as a trivial subclass of the Python Exception class.

```
class Empty(Exception):
    """Error attempting to access an element from an empty container."""
    pass
```

Code Fragment 6.1: Definition for an `Empty` exception class.

The formal definition for our `ArrayStack` class is given in Code Fragment 6.2. The constructor establishes the member `self._data` as an initially empty Python list, for internal storage. The rest of the public stack behaviors are implemented, using the corresponding adaptation that was outlined in Table 6.1.

Example Usage

Below, we present an example of the use of our `ArrayStack` class, mirroring the operations at the beginning of Example 6.3 on page 230.

<code>S = ArrayStack()</code>	<code># contents: []</code>	
<code>S.push(5)</code>	<code># contents: [5]</code>	
<code>S.push(3)</code>	<code># contents: [5, 3]</code>	
<code>print(len(S))</code>	<code># contents: [5, 3];</code>	outputs 2
<code>print(S.pop())</code>	<code># contents: [5];</code>	outputs 3
<code>print(S.is_empty())</code>	<code># contents: [5];</code>	outputs False
<code>print(S.pop())</code>	<code># contents: [];</code>	outputs 5
<code>print(S.is_empty())</code>	<code># contents: [];</code>	outputs True
<code>S.push(7)</code>	<code># contents: [7]</code>	
<code>S.push(9)</code>	<code># contents: [7, 9]</code>	
<code>print(S.top())</code>	<code># contents: [7, 9];</code>	outputs 9
<code>S.push(4)</code>	<code># contents: [7, 9, 4]</code>	
<code>print(len(S))</code>	<code># contents: [7, 9, 4];</code>	outputs 3
<code>print(S.pop())</code>	<code># contents: [7, 9];</code>	outputs 4
<code>S.push(6)</code>	<code># contents: [7, 9, 6]</code>	

```

1  class ArrayStack:
2      """ LIFO Stack implementation using a Python list as underlying storage."""
3
4      def __init__(self):
5          """ Create an empty stack."""
6          self._data = [ ]                # nonpublic list instance
7
8      def __len__(self):
9          """ Return the number of elements in the stack."""
10         return len(self._data)
11
12     def is_empty(self):
13         """ Return True if the stack is empty."""
14         return len(self._data) == 0
15
16     def push(self, e):
17         """ Add element e to the top of the stack."""
18         self._data.append(e)            # new item stored at end of list
19
20     def top(self):
21         """ Return (but do not remove) the element at the top of the stack.
22
23         Raise Empty exception if the stack is empty.
24         """
25         if self.is_empty():
26             raise Empty('Stack is empty')
27         return self._data[-1]           # the last item in the list
28
29     def pop(self):
30         """ Remove and return the element from the top of the stack (i.e., LIFO).
31
32         Raise Empty exception if the stack is empty.
33         """
34         if self.is_empty():
35             raise Empty('Stack is empty')
36         return self._data.pop( )        # remove last item from list

```

Code Fragment 6.2: Implementing a stack using a Python list as storage.

Analyzing the Array-Based Stack Implementation

Table 6.2 shows the running times for our `ArrayStack` methods. The analysis directly mirrors the analysis of the list class given in Section 5.3. The implementations for `top`, `is_empty`, and `len` use constant time in the worst case. The $O(1)$ time for `push` and `pop` are *amortized* bounds (see Section 5.3.2); a typical call to either of these methods uses constant time, but there is occasionally an $O(n)$ -time worst case, where n is the current number of elements in the stack, when an operation causes the list to resize its internal array. The space usage for a stack is $O(n)$.

Operation	Running Time
<code>S.push(e)</code>	$O(1)^*$
<code>S.pop()</code>	$O(1)^*$
<code>S.top()</code>	$O(1)$
<code>S.is_empty()</code>	$O(1)$
<code>len(S)</code>	$O(1)$

*amortized

Table 6.2: Performance of our array-based stack implementation. The bounds for `push` and `pop` are amortized due to similar bounds for the list class. The space usage is $O(n)$, where n is the current number of elements in the stack.

Avoiding Amortization by Reserving Capacity

In some contexts, there may be additional knowledge that suggests a maximum size that a stack will reach. Our implementation of `ArrayStack` from Code Fragment 6.2 begins with an empty list and expands as needed. In the analysis of lists from Section 5.4.1, we emphasized that it is more efficient in practice to construct a list with initial length n than it is to start with an empty list and append n items (even though both approaches run in $O(n)$ time).

As an alternate model for a stack, we might wish for the constructor to accept a parameter specifying the maximum capacity of a stack and to initialize the `_data` member to a list of that length. Implementing such a model requires significant changes relative to Code Fragment 6.2. The size of the stack would no longer be synonymous with the length of the list, and pushes and pops of the stack would not require changing the length of the list. Instead, we suggest maintaining a separate integer as an instance variable that denotes the current number of elements in the stack. Details of such an implementation are left as Exercise C-6.17.

6.1.3 Reversing Data Using a Stack

As a consequence of the LIFO protocol, a stack can be used as a general tool to reverse a data sequence. For example, if the values 1, 2, and 3 are pushed onto a stack in that order, they will be popped from the stack in the order 3, 2, and then 1.

This idea can be applied in a variety of settings. For example, we might wish to print lines of a file in reverse order in order to display a data set in decreasing order rather than increasing order. This can be accomplished by reading each line and pushing it onto a stack, and then writing the lines in the order they are popped. An implementation of such a process is given in Code Fragment 6.3.

```
1 def reverse_file(filename):
2     """ Overwrite given file with its contents line-by-line reversed."""
3     S = ArrayStack()
4     original = open(filename)
5     for line in original:
6         S.push(line.rstrip('\n'))      # we will re-insert newlines when writing
7     original.close()
8
9     # now we overwrite with contents in LIFO order
10    output = open(filename, 'w')      # reopening file overwrites original
11    while not S.is_empty():
12        output.write(S.pop() + '\n')  # re-insert newline characters
13    output.close()
```

Code Fragment 6.3: A function that reverses the order of lines in a file.

One technical detail worth noting is that we intentionally strip trailing newlines from lines as they are read, and then re-insert newlines after each line when writing the resulting file. Our reason for doing this is to handle a special case in which the original file does not have a trailing newline for the final line. If we exactly echoed the lines read from the file in reverse order, then the original last line would be followed (without newline) by the original second-to-last line. In our implementation, we ensure that there will be a separating newline in the result.

The idea of using a stack to reverse a data set can be applied to other types of sequences. For example, Exercise R-6.5 explores the use of a stack to provide yet another solution for reversing the contents of a Python list (a recursive solution for this goal was discussed in Section 4.4.1). A more challenging task is to reverse the order in which elements are stored within a stack. If we were to move them from one stack to another, they would be reversed, but if we were to then replace them into the original stack, they would be reversed again, thereby reverting to their original order. Exercise C-6.18 explores a solution for this task.

6.1.4 Matching Parentheses and HTML Tags

In this subsection, we explore two related applications of stacks, both of which involve testing for pairs of matching delimiters. In our first application, we consider arithmetic expressions that may contain various pairs of grouping symbols, such as

- Parentheses: “(” and “)”
- Braces: “{” and “}”
- Brackets: “[” and “]”

Each opening symbol must match its corresponding closing symbol. For example, a left bracket, “[,” must match a corresponding right bracket, “],” as in the expression $[(5+x)-(y+z)]$. The following examples further illustrate this concept:

- Correct: $()(())\{([()])\}$
- Correct: $((())(())\{([()])\})$
- Incorrect: $)()()\{([()])\}$
- Incorrect: $\{([()])\}$
- Incorrect: $($

We leave the precise definition of a matching group of symbols to Exercise R-6.6.

An Algorithm for Matching Delimiters

An important task when processing arithmetic expressions is to make sure their delimiting symbols match up correctly. Code Fragment 6.4 presents a Python implementation of such an algorithm. A discussion of the code follows.

```

1  def is_matched(expr):
2      """Return True if all delimiters are properly match; False otherwise."""
3      lefty = '([{'                                     # opening delimiters
4      righty = ')]}'                                    # respective closing delims
5      S = ArrayStack()
6      for c in expr:
7          if c in lefty:
8              S.push(c)                                  # push left delimiter on stack
9          elif c in righty:
10             if S.is_empty():
11                 return False                            # nothing to match with
12             if righty.index(c) != lefty.index(S.pop()):
13                 return False                            # mismatched
14         return S.is_empty()                             # were all symbols matched?
```

Code Fragment 6.4: Function for matching delimiters in an arithmetic expression.

We assume the input is a sequence of characters, such as `'[(5+x)-(y+z)]'`. We perform a left-to-right scan of the original sequence, using a stack S to facilitate the matching of grouping symbols. Each time we encounter an opening symbol, we push that symbol onto S , and each time we encounter a closing symbol, we pop a symbol from the stack S (assuming S is not empty), and check that these two symbols form a valid pair. If we reach the end of the expression and the stack is empty, then the original expression was properly matched. Otherwise, there must be an opening delimiter on the stack without a matching symbol.

If the length of the original expression is n , the algorithm will make at most n calls to push and n calls to pop. Those calls run in a total of $O(n)$ time, even considering the amortized nature of the $O(1)$ time bound for those methods. Given that our selection of possible delimiters, $\{[, [$, has constant size, auxiliary tests such as `c in lefty` and `righty.index(c)` each run in $O(1)$ time. Combining these operations, the matching algorithm on a sequence of length n runs in $O(n)$ time.

Matching Tags in a Markup Language

Another application of matching delimiters is in the validation of markup languages such as HTML or XML. HTML is the standard format for hyperlinked documents on the Internet and XML is an extensible markup language used for a variety of structured data sets. We show a sample HTML document and a possible rendering in Figure 6.3.

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Figure 6.3: Illustrating HTML tags. (a) An HTML document; (b) its rendering.

In an HTML document, portions of text are delimited by *HTML tags*. A simple opening HTML tag has the form “<name>” and the corresponding closing tag has the form “</name>”. For example, we see the <body> tag on the first line of Figure 6.3(a), and the matching </body> tag at the close of that document. Other commonly used HTML tags that are used in this example include:

- body: document body
- h1: section header
- center: center justify
- p: paragraph
- ol: numbered (ordered) list
- li: list item

Ideally, an HTML document should have matching tags, although most browsers tolerate a certain number of mismatching tags. In Code Fragment 6.5, we give a Python function that matches tags in a string representing an HTML document. We make a left-to-right pass through the raw string, using index j to track our progress and the find method of the str class to locate the ‘<’ and ‘>’ characters that define the tags. Opening tags are pushed onto the stack, and matched against closing tags as they are popped from the stack, just as we did when matching delimiters in Code Fragment 6.4. By similar analysis, this algorithm runs in $O(n)$ time, where n is the number of characters in the raw HTML source.

```

1  def is_matched_html(raw):
2      """Return True if all HTML tags are properly match; False otherwise."""
3      S = ArrayStack()
4      j = raw.find('<')                # find first '<' character (if any)
5      while j != -1:
6          k = raw.find('>', j+1)      # find next '>' character
7          if k == -1:
8              return False          # invalid tag
9          tag = raw[j+1:k]           # strip away < >
10         if not tag.startswith('/'): # this is opening tag
11             S.push(tag)
12         else:                       # this is closing tag
13             if S.is_empty():
14                 return False       # nothing to match with
15             if tag[1:] != S.pop():
16                 return False       # mismatched delimiter
17             j = raw.find('<', k+1)   # find next '<' character (if any)
18     return S.is_empty()            # were all opening tags matched?

```

Code Fragment 6.5: Function for testing if an HTML document has matching tags.

6.2 Queues

Another fundamental data structure is the *queue*. It is a close “cousin” of the stack, as a queue is a collection of objects that are inserted and removed according to the *first-in, first-out (FIFO)* principle. That is, elements can be inserted at any time, but only the element that has been in the queue the longest can be next removed.

We usually say that elements enter a queue at the back and are removed from the front. A metaphor for this terminology is a line of people waiting to get on an amusement park ride. People waiting for such a ride enter at the back of the line and get on the ride from the front of the line. There are many other applications of queues (see Figure 6.4). Stores, theaters, reservation centers, and other similar services typically process customer requests according to the FIFO principle. A queue would therefore be a logical choice for a data structure to handle calls to a customer service center, or a wait-list at a restaurant. FIFO queues are also used by many computing devices, such as a networked printer, or a Web server responding to requests.

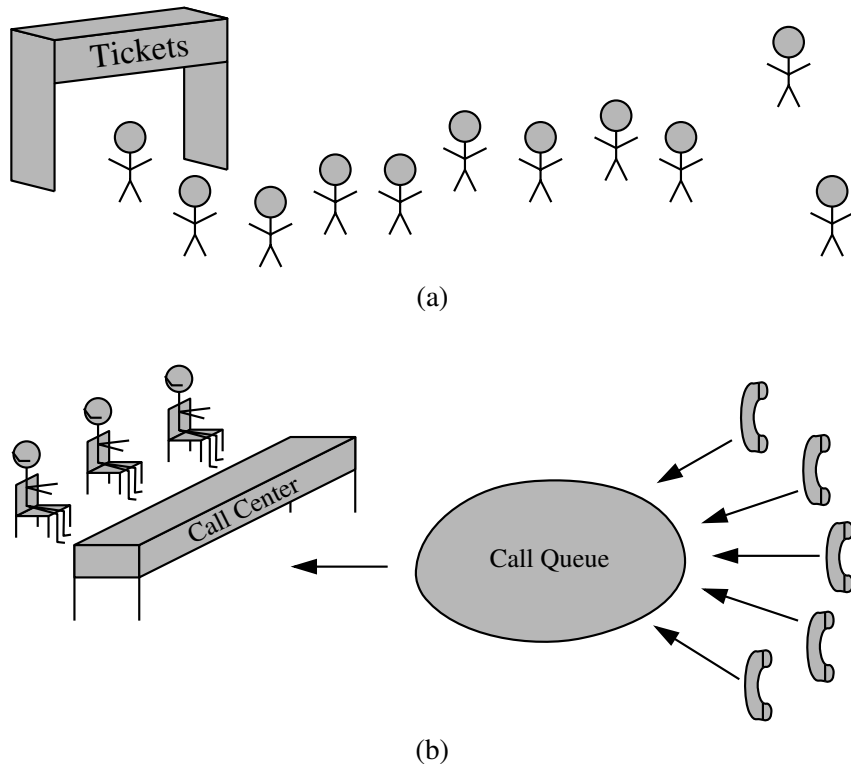


Figure 6.4: Real-world examples of a first-in, first-out queue. (a) People waiting in line to purchase tickets; (b) phone calls being routed to a customer service center.

6.2.1 The Queue Abstract Data Type

Formally, the queue abstract data type defines a collection that keeps objects in a sequence, where element access and deletion are restricted to the *first* element in the queue, and element insertion is restricted to the back of the sequence. This restriction enforces the rule that items are inserted and deleted in a queue according to the first-in, first-out (FIFO) principle. The *queue* abstract data type (ADT) supports the following two fundamental methods for a queue *Q*:

Q.enqueue(e): Add element *e* to the back of queue *Q*.

Q.dequeue(): Remove and return the first element from queue *Q*;
an error occurs if the queue is empty.

The queue ADT also includes the following supporting methods (with first being analogous to the stack's top method):

Q.first(): Return a reference to the element at the front of queue *Q*,
without removing it; an error occurs if the queue is empty.

Q.is_empty(): Return True if queue *Q* does not contain any elements.

len(Q): Return the number of elements in queue *Q*; in Python,
we implement this with the special method `__len__`.

By convention, we assume that a newly created queue is empty, and that there is no a priori bound on the capacity of the queue. Elements added to the queue can have arbitrary type.

Example 6.4: The following table shows a series of queue operations and their effects on an initially empty queue *Q* of integers.

Operation	Return Value	first $\leftarrow Q \leftarrow$ last
Q.enqueue(5)	—	[5]
Q.enqueue(3)	—	[5, 3]
len(Q)	2	[5, 3]
Q.dequeue()	5	[3]
Q.is_empty()	False	[3]
Q.dequeue()	3	[]
Q.is_empty()	True	[]
Q.dequeue()	“error”	[]
Q.enqueue(7)	—	[7]
Q.enqueue(9)	—	[7, 9]
Q.first()	7	[7, 9]
Q.enqueue(4)	—	[7, 9, 4]
len(Q)	3	[7, 9, 4]
Q.dequeue()	7	[9, 4]

6.2.2 Array-Based Queue Implementation

For the stack ADT, we created a very simple adapter class that used a Python list as the underlying storage. It may be very tempting to use a similar approach for supporting the queue ADT. We could enqueue element e by calling `append(e)` to add it to the end of the list. We could use the syntax `pop(0)`, as opposed to `pop()`, to intentionally remove the *first* element from the list when dequeuing.

As easy as this would be to implement, it is tragically inefficient. As we discussed in Section 5.4.1, when `pop` is called on a list with a non-default index, a loop is executed to shift all elements beyond the specified index to the left, so as to fill the hole in the sequence caused by the `pop`. Therefore, a call to `pop(0)` always causes the worst-case behavior of $\Theta(n)$ time.

We can improve on the above strategy by avoiding the call to `pop(0)` entirely. We can replace the dequeued entry in the array with a reference to `None`, and maintain an explicit variable f to store the index of the element that is currently at the front of the queue. Such an algorithm for dequeue would run in $O(1)$ time. After several dequeue operations, this approach might lead to the configuration portrayed in Figure 6.5.

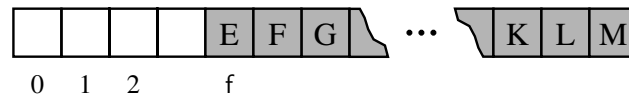


Figure 6.5: Allowing the front of the queue to drift away from index 0.

Unfortunately, there remains a drawback to the revised approach. In the case of a stack, the length of the list was precisely equal to the size of the stack (even if the underlying array for the list was slightly larger). With the queue design that we are considering, the situation is worse. We can build a queue that has relatively few elements, yet which are stored in an arbitrarily large list. This occurs, for example, if we repeatedly enqueue a new element and then dequeue another (allowing the front to drift rightward). Over time, the size of the underlying list would grow to $O(m)$ where m is the *total* number of enqueue operations since the creation of the queue, rather than the current number of elements in the queue.

This design would have detrimental consequences in applications in which queues have relatively modest size, but which are used for long periods of time. For example, the wait-list for a restaurant might never have more than 30 entries at one time, but over the course of a day (or a week), the overall number of entries would be significantly larger.

Using an Array Circularly

In developing a more robust queue implementation, we allow the front of the queue to drift rightward, and we allow the contents of the queue to “wrap around” the end of an underlying array. We assume that our underlying array has fixed length N that is greater than the actual number of elements in the queue. New elements are enqueued toward the “end” of the current queue, progressing from the front to index $N - 1$ and continuing at index 0, then 1. Figure 6.6 illustrates such a queue with first element E and last element M.



Figure 6.6: Modeling a queue with a circular array that wraps around the end.

Implementing this circular view is not difficult. When we dequeue an element and want to “advance” the front index, we use the arithmetic $f = (f + 1) \% N$. Recall that the `%` operator in Python denotes the *modulo* operator, which is computed by taking the remainder after an integral division. For example, 14 divided by 3 has a quotient of 4 with remainder 2, that is, $\frac{14}{3} = 4\frac{2}{3}$. So in Python, `14 // 3` evaluates to the quotient 4, while `14 % 3` evaluates to the remainder 2. The modulo operator is ideal for treating an array circularly. As a concrete example, if we have a list of length 10, and a front index 7, we can advance the front by formally computing $(7+1) \% 10$, which is simply 8, as 8 divided by 10 is 0 with a remainder of 8. Similarly, advancing index 8 results in index 9. But when we advance from index 9 (the last one in the array), we compute $(9+1) \% 10$, which evaluates to index 0 (as 10 divided by 10 has a remainder of zero).

A Python Queue Implementation

A complete implementation of a queue ADT using a Python list in circular fashion is presented in Code Fragments 6.6 and 6.7. Internally, the queue class maintains the following three instance variables:

- `_data`:** is a reference to a list instance with a fixed capacity.
- `_size`:** is an integer representing the current number of elements stored in the queue (as opposed to the length of the `_data` list).
- `_front`:** is an integer that represents the index within `_data` of the first element of the queue (assuming the queue is not empty).

We initially reserve a list of moderate size for storing data, although the queue formally has size zero. As a technicality, we initialize the `_front` index to zero.

When `front` or `dequeue` are called with no elements in the queue, we raise an instance of the `Empty` exception, defined in Code Fragment 6.1 for our stack.

```

1  class ArrayQueue:
2      """ FIFO queue implementation using a Python list as underlying storage."""
3      DEFAULT_CAPACITY = 10          # moderate capacity for all new queues
4
5      def __init__(self):
6          """ Create an empty queue."""
7          self._data = [None] * ArrayQueue.DEFAULT_CAPACITY
8          self._size = 0
9          self._front = 0
10
11     def __len__(self):
12         """ Return the number of elements in the queue."""
13         return self._size
14
15     def is_empty(self):
16         """ Return True if the queue is empty."""
17         return self._size == 0
18
19     def first(self):
20         """ Return (but do not remove) the element at the front of the queue.
21
22         Raise Empty exception if the queue is empty.
23         """
24         if self.is_empty():
25             raise Empty('Queue is empty')
26         return self._data[self._front]
27
28     def dequeue(self):
29         """ Remove and return the first element of the queue (i.e., FIFO).
30
31         Raise Empty exception if the queue is empty.
32         """
33         if self.is_empty():
34             raise Empty('Queue is empty')
35         answer = self._data[self._front]
36         self._data[self._front] = None          # help garbage collection
37         self._front = (self._front + 1) % len(self._data)
38         self._size -= 1
39         return answer

```

Code Fragment 6.6: Array-based implementation of a queue (continued in Code Fragment 6.7).

```

40 def enqueue(self, e):
41     """Add an element to the back of queue."""
42     if self._size == len(self._data):
43         self._resize(2 * len(self._data))    # double the array size
44     avail = (self._front + self._size) % len(self._data)
45     self._data[avail] = e
46     self._size += 1
47
48 def _resize(self, cap):                      # we assume cap >= len(self)
49     """Resize to a new list of capacity >= len(self)."""
50     old = self._data                        # keep track of existing list
51     self._data = [None] * cap              # allocate list with new capacity
52     walk = self._front
53     for k in range(self._size):             # only consider existing elements
54         self._data[k] = old[walk]          # intentionally shift indices
55         walk = (1 + walk) % len(old)        # use old size as modulus
56     self._front = 0                        # front has been realigned

```

Code Fragment 6.7: Array-based implementation of a queue (continued from Code Fragment 6.6).

The implementation of `__len__` and `is_empty` are trivial, given knowledge of the size. The implementation of the `front` method is also simple, as the `_front` index tells us precisely where the desired element is located within the `_data` list, assuming that list is not empty.

Adding and Removing Elements

The goal of the `enqueue` method is to add a new element to the back of the queue. We need to determine the proper index at which to place the new element. Although we do not explicitly maintain an instance variable for the back of the queue, we compute the location of the next opening based on the formula:

$$\text{avail} = (\text{self._front} + \text{self._size}) \% \text{len}(\text{self._data})$$

Note that we are using the size of the queue as it exists *prior* to the addition of the new element. For example, consider a queue with capacity 10, current size 3, and first element at index 5. The three elements of such a queue are stored at indices 5, 6, and 7. The new element should be placed at index $(\text{front} + \text{size}) = 8$. In a case with wrap-around, the use of the modular arithmetic achieves the desired circular semantics. For example, if our hypothetical queue had 3 elements with the first at index 8, our computation of $(8+3) \% 10$ evaluates to 1, which is perfect since the three existing elements occupy indices 8, 9, and 0.

When the `dequeue` method is called, the current value of `self._front` designates the index of the value that is to be removed and returned. We keep a local reference to the element that will be returned, setting `answer = self._data[self._front]` just prior to removing the reference to that object from the list, with the assignment `self._data[self._front] = None`. Our reason for the assignment to `None` relates to Python's mechanism for reclaiming unused space. Internally, Python maintains a count of the number of references that exist to each object. If that count reaches zero, the object is effectively inaccessible, thus the system may reclaim that memory for future use. (For more details, see Section 15.1.2.) Since we are no longer responsible for storing a dequeued element, we remove the reference to it from our list so as to reduce that element's reference count.

The second significant responsibility of the `dequeue` method is to update the value of `_front` to reflect the removal of the element, and the presumed promotion of the second element to become the new first. In most cases, we simply want to increment the index by one, but because of the possibility of a wrap-around configuration, we rely on modular arithmetic as originally described on page 242.

Resizing the Queue

When `enqueue` is called at a time when the size of the queue equals the size of the underlying list, we rely on a standard technique of doubling the storage capacity of the underlying list. In this way, our approach is similar to the one used when we implemented a `DynamicArray` in Section 5.3.1.

However, more care is needed in the queue's `_resize` utility than was needed in the corresponding method of the `DynamicArray` class. After creating a temporary reference to the old list of values, we allocate a new list that is twice the size and copy references from the old list to the new list. While transferring the contents, we intentionally realign the front of the queue with index 0 in the new array, as shown in Figure 6.7. This realignment is not purely cosmetic. Since the modular arithmetic depends on the size of the array, our state would be flawed had we transferred each element to its same index in the new array.

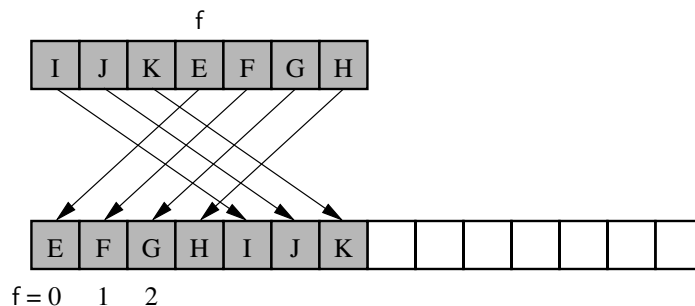


Figure 6.7: Resizing the queue, while realigning the front element with index 0.

Shrinking the Underlying Array

A desirable property of a queue implementation is to have its space usage be $\Theta(n)$ where n is the current number of elements in the queue. Our `ArrayQueue` implementation, as given in Code Fragments 6.6 and 6.7, does not have this property. It expands the underlying array when `enqueue` is called with the queue at full capacity, but the `dequeue` implementation never shrinks the underlying array. As a consequence, the capacity of the underlying array is proportional to the maximum number of elements that have ever been stored in the queue, not the current number of elements.

We discussed this very issue on page 200, in the context of dynamic arrays, and in subsequent Exercises C-5.16 through C-5.20 of that chapter. A robust approach is to reduce the array to half of its current size, whenever the number of elements stored in it falls below *one fourth* of its capacity. We can implement this strategy by adding the following two lines of code in our `dequeue` method, just after reducing `self._size` at line 38 of Code Fragment 6.6, to reflect the loss of an element.

```
if 0 < self._size < len(self._data) // 4:
    self._resize(len(self._data) // 2)
```

Analyzing the Array-Based Queue Implementation

Table 6.3 describes the performance of our array-based implementation of the queue ADT, assuming the improvement described above for occasionally shrinking the size of the array. With the exception of the `_resize` utility, all of the methods rely on a constant number of statements involving arithmetic operations, comparisons, and assignments. Therefore, each method runs in worst-case $O(1)$ time, except for `enqueue` and `dequeue`, which have *amortized* bounds of $O(1)$ time, for reasons similar to those given in Section 5.3.

Operation	Running Time
<code>Q.enqueue(e)</code>	$O(1)^*$
<code>Q.dequeue()</code>	$O(1)^*$
<code>Q.first()</code>	$O(1)$
<code>Q.is_empty()</code>	$O(1)$
<code>len(Q)</code>	$O(1)$

*amortized

Table 6.3: Performance of an array-based implementation of a queue. The bounds for `enqueue` and `dequeue` are amortized due to the resizing of the array. The space usage is $O(n)$, where n is the current number of elements in the queue.

6.3 Double-Ended Queues

We next consider a queue-like data structure that supports insertion and deletion at both the front and the back of the queue. Such a structure is called a ***double-ended queue***, or ***deque***, which is usually pronounced “deck” to avoid confusion with the dequeue method of the regular queue ADT, which is pronounced like the abbreviation “D.Q.”

The deque abstract data type is more general than both the stack and the queue ADTs. The extra generality can be useful in some applications. For example, we described a restaurant using a queue to maintain a waitlist. Occasionally, the first person might be removed from the queue only to find that a table was not available; typically, the restaurant will re-insert the person at the *first* position in the queue. It may also be that a customer at the end of the queue may grow impatient and leave the restaurant. (We will need an even more general data structure if we want to model customers leaving the queue from other positions.)

6.3.1 The Deque Abstract Data Type

To provide a symmetrical abstraction, the deque ADT is defined so that deque D supports the following methods:

D.add_first(e): Add element e to the front of deque D.

D.add_last(e): Add element e to the back of deque D.

D.delete_first(): Remove and return the first element from deque D;
an error occurs if the deque is empty.

D.delete_last(): Remove and return the last element from deque D;
an error occurs if the deque is empty.

Additionally, the deque ADT will include the following accessors:

D.first(): Return (but do not remove) the first element of deque D;
an error occurs if the deque is empty.

D.last(): Return (but do not remove) the last element of deque D;
an error occurs if the deque is empty.

D.is_empty(): Return True if deque D does not contain any elements.

len(D): Return the number of elements in deque D; in Python,
we implement this with the special method `__len__`.

Example 6.5: The following table shows a series of operations and their effects on an initially empty deque *D* of integers.

Operation	Return Value	Deque
D.add_last(5)	–	[5]
D.add_first(3)	–	[3, 5]
D.add_first(7)	–	[7, 3, 5]
D.first()	7	[7, 3, 5]
D.delete_last()	5	[7, 3]
len(D)	2	[7, 3]
D.delete_last()	3	[7]
D.delete_last()	7	[]
D.add_first(6)	–	[6]
D.last()	6	[6]
D.add_first(8)	–	[8, 6]
D.is_empty()	False	[8, 6]
D.last()	6	[8, 6]

6.3.2 Implementing a Deque with a Circular Array

We can implement the deque ADT in much the same way as the `ArrayQueue` class provided in Code Fragments 6.6 and 6.7 of Section 6.2.2 (so much so that we leave the details of an `ArrayDeque` implementation to Exercise P-6.32). We recommend maintaining the same three instance variables: `_data`, `_size`, and `_front`. Whenever we need to know the index of the back of the deque, or the first available slot beyond the back of the deque, we use modular arithmetic for the computation. For example, our implementation of the `last()` method uses the index

```
back = (self._front + self._size - 1) % len(self._data)
```

Our implementation of the `ArrayDeque.add_last` method is essentially the same as that for `ArrayQueue.enqueue`, including the reliance on a `_resize` utility. Likewise, the implementation of the `ArrayDeque.delete_first` method is the same as `ArrayQueue.dequeue`. Implementations of `add_first` and `delete_last` use similar techniques. One subtlety is that a call to `add_first` may need to wrap around the beginning of the array, so we rely on modular arithmetic to circularly *decrement* the index, as

```
self._front = (self._front - 1) % len(self._data)           # cyclic shift
```

The efficiency of an `ArrayDeque` is similar to that of an `ArrayQueue`, with all operations having $O(1)$ running time, but with that bound being amortized for operations that may change the size of the underlying list.

6.3.3 Deques in the Python Collections Module

An implementation of a deque class is available in Python's standard collections module. A summary of the most commonly used behaviors of the collections.deque class is given in Table 6.4. It uses more asymmetric nomenclature than our ADT.

Our Deque ADT	collections.deque	Description
len(D)	len(D)	number of elements
D.add_first()	D.appendleft()	add to beginning
D.add_last()	D.append()	add to end
D.delete_first()	D.popleft()	remove from beginning
D.delete_last()	D.pop()	remove from end
D.first()	D[0]	access first element
D.last()	D[-1]	access last element
	D[j]	access arbitrary entry by index
	D[j] = val	modify arbitrary entry by index
	D.clear()	clear all contents
	D.rotate(k)	circularly shift rightward k steps
	D.remove(e)	remove first matching element
	D.count(e)	count number of matches for e

Table 6.4: Comparison of our deque ADT and the collections.deque class.

The collections.deque interface was chosen to be consistent with established naming conventions of Python's list class, for which append and pop are presumed to act at the end of the list. Therefore, appendleft and popleft designate an operation at the beginning of the list. The library deque also mimics a list in that it is an indexed sequence, allowing arbitrary access or modification using the D[j] syntax.

The library deque constructor also supports an optional maxlen parameter to force a fixed-length deque. However, if a call to append at either end is invoked when the deque is full, it does not throw an error; instead, it causes one element to be dropped from the opposite side. That is, calling appendleft when the deque is full causes an implicit pop from the right side to make room for the new element.

The current Python distribution implements collections.deque with a hybrid approach that uses aspects of circular arrays, but organized into blocks that are themselves organized in a doubly linked list (a data structure that we will introduce in the next chapter). The deque class is formally documented to guarantee $O(1)$ -time operations at either end, but $O(n)$ -time worst-case operations when using index notation near the middle of the deque.

Object References, Mutability, and Recycling

“You are sad,” the Knight said in an anxious tone: “let me sing you a song to comfort you. [...] The name of the song is called ‘HADDOCKS’ EYES’.”

“Oh, that’s the name of the song, is it?” Alice said, trying to feel interested.

“No, you don’t understand,” the Knight said, looking a little vexed. “That’s what the name is CALLED. The name really IS ‘THE AGED AGED MAN.’”

—Adapted from Lewis Carroll, *Through the Looking-Glass, and What Alice Found There*

Alice and the Knight set the tone of what we will see in this chapter. The theme is the distinction between objects and their names. A name is not the object; a name is a separate thing.

We start the chapter by presenting a metaphor for variables in Python: variables are labels, not boxes. If reference variables are old news to you, the analogy may still be handy if you need to explain aliasing issues to others.

We then discuss the concepts of object identity, value, and aliasing. A surprising trait of tuples is revealed: they are immutable but their values may change. This leads to a discussion of shallow and deep copies. References and function parameters are our next theme: the problem with mutable parameter defaults and the safe handling of mutable arguments passed by clients of our functions.

The last sections of the chapter cover garbage collection, the `del` command, and a selection of tricks that Python plays with immutable objects.

This is a rather dry chapter, but its topics lie at the heart of many subtle bugs in real Python programs.

What's New in This Chapter

The topics covered here are very fundamental and stable. There were no changes worth mentioning in this second edition.

I added an example of using `is` to test for a sentinel object, and a warning about misuses of the `is` operator at the end of “[Choosing Between `==` and `is`](#)” on page 206.

This chapter used to be in Part IV, but I decided to bring it up earlier because it works better as an ending to Part II, “Data Structures,” than an opening to “Object-Oriented Idioms.”



The section on “Weak References” from the first edition of this book is now a [post at *fluentpython.com*](#).

Let's start by unlearning that a variable is like a box where you store data.

Variables Are Not Boxes

In 1997, I took a summer course on Java at MIT. The professor, Lynn Stein,¹ made the point that the usual “variables as boxes” metaphor actually hinders the understanding of reference variables in object-oriented languages. Python variables are like reference variables in Java; a better metaphor is to think of variables as labels with names attached to objects. The next example and figure will help you understand why.

Example 6-1 is a simple interaction that the “variables as boxes” idea cannot explain. **Figure 6-1** illustrates why the box metaphor is wrong for Python, while sticky notes provide a helpful picture of how variables actually work.

Example 6-1. Variables `a` and `b` hold references to the same list, not copies of the list

```
>>> a = [1, 2, 3] ①
>>> b = a         ②
>>> a.append(4)    ③
>>> b              ④
[1, 2, 3, 4]
```

¹ Lynn Andrea Stein is an award-winning computer science educator who [currently teaches at Olin College of Engineering](#).

- ❶ Create a list `[1, 2, 3]` and bind the variable `a` to it.
- ❷ Bind the variable `b` to the same value that `a` is referencing.
- ❸ Modify the list referenced by `a`, by appending another item.
- ❹ You can see the effect via the `b` variable. If we think of `b` as a box that stored a copy of the `[1, 2, 3]` from the `a` box, this behavior makes no sense.

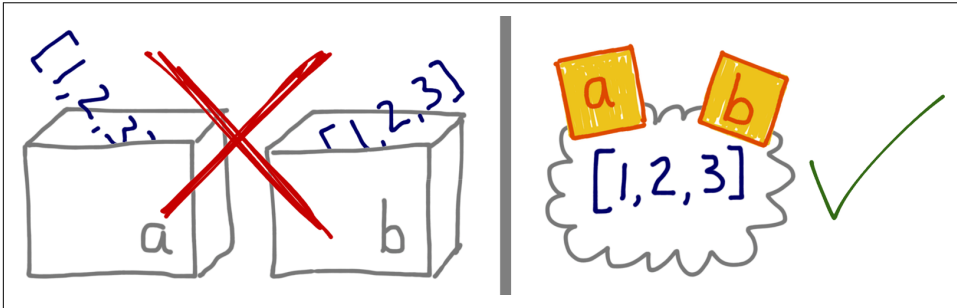


Figure 6-1. If you imagine variables are like boxes, you can’t make sense of assignment in Python; instead, think of variables as sticky notes, and [Example 6-1](#) becomes easy to explain.

Therefore, the `b = a` statement does not copy the contents of box `a` into box `b`. It attaches the label `b` to the object that already has the label `a`.

Prof. Stein also spoke about assignment in a very deliberate way. For example, when talking about a seesaw object in a simulation, she would say: “Variable `s` is assigned to the seesaw,” but never “The seesaw is assigned to variable `s`.” With reference variables, it makes much more sense to say that the variable is assigned to an object, and not the other way around. After all, the object is created before the assignment. [Example 6-2](#) proves that the righthand side of an assignment happens first.

Since the verb “to assign” is used in contradictory ways, a useful alternative is “to bind”: Python’s assignment statement `x = ...` binds the `x` name to the object created or referenced on the righthand side. And the object must exist before a name can be bound to it, as [Example 6-2](#) proves.

Example 6-2. Variables are bound to objects only after the objects are created

```
>>> class Gizmo:
...     def __init__(self):
...         print(f'Gizmo id: {id(self)}')
...
>>> x = Gizmo()
```



```

Gizmo id: 4301489152 ❶
>>> y = Gizmo() * 10 ❷
Gizmo id: 4301489432 ❸
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for *: 'Gizmo' and 'int'
>>>
>>> dir() ❹
['Gizmo', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'x']

```

- ❶ The output `Gizmo id: ...` is a side effect of creating a `Gizmo` instance.
- ❷ Multiplying a `Gizmo` instance will raise an exception.
- ❸ Here is proof that a second `Gizmo` was actually instantiated before the multiplication was attempted.
- ❹ But variable `y` was never created, because the exception happened while the righthand side of the assignment was being evaluated.



To understand an assignment in Python, read the righthand side first: that's where the object is created or retrieved. After that, the variable on the left is bound to the object, like a label stuck to it. Just forget about the boxes.

Because variables are mere labels, nothing prevents an object from having several labels assigned to it. When that happens, you have *aliasing*, our next topic.

Identity, Equality, and Aliases

Lewis Carroll is the pen name of Prof. Charles Lutwidge Dodgson. Mr. Carroll is not only equal to Prof. Dodgson, they are one and the same. **Example 6-3** expresses this idea in Python.

Example 6-3. `charles` and `lewis` refer to the same object

```

>>> charles = {'name': 'Charles L. Dodgson', 'born': 1832}
>>> lewis = charles ❶
>>> lewis is charles
True
>>> id(charles), id(lewis) ❷
(4300473992, 4300473992)
>>> lewis['balance'] = 950 ❸

```

```
>>> charles
{'name': 'Charles L. Dodgson', 'born': 1832, 'balance': 950}
```

- ❶ `lewis` is an alias for `charles`.
- ❷ The `is` operator and the `id` function confirm it.
- ❸ Adding an item to `lewis` is the same as adding an item to `charles`.

However, suppose an impostor—let’s call him Dr. Alexander Pedachenko—claims he is Charles L. Dodgson, born in 1832. His credentials may be the same, but Dr. Pedachenko is not Prof. Dodgson. Figure 6-2 illustrates this scenario.

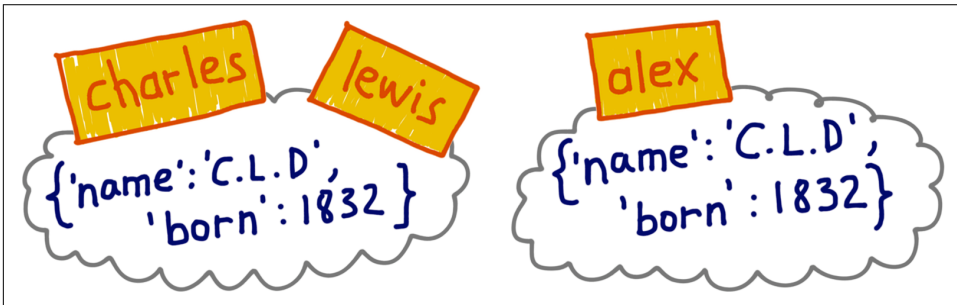


Figure 6-2. `charles` and `lewis` are bound to the same object; `alex` is bound to a separate object of equal value.

Example 6-4 implements and tests the `alex` object depicted in Figure 6-2.

Example 6-4. `alex` and `charles` compare equal, but `alex` is not `charles`

```
>>> alex = {'name': 'Charles L. Dodgson', 'born': 1832, 'balance': 950} ❶
>>> alex == charles ❷
True
>>> alex is not charles ❸
True
```

- ❶ `alex` refers to an object that is a replica of the object assigned to `charles`.
- ❷ The objects compare equal because of the `__eq__` implementation in the `dict` class.
- ❸ But they are distinct objects. This is the Pythonic way of writing the negative identity comparison: `a is not b`.

Example 6-3 is an example of *aliasing*. In that code, `lewis` and `charles` are aliases: two variables bound to the same object. On the other hand, `alex` is not an alias for

charles: these variables are bound to distinct objects. The objects bound to alex and charles have the same *value*—that’s what `==` compares—but they have different identities.

In *The Python Language Reference*, “3.1. Objects, values and types” states:

An object’s identity never changes once it has been created; you may think of it as the object’s address in memory. The `is` operator compares the identity of two objects; the `id()` function returns an integer representing its identity.

The real meaning of an object’s ID is implementation dependent. In CPython, `id()` returns the memory address of the object, but it may be something else in another Python interpreter. The key point is that the ID is guaranteed to be a unique integer label, and it will never change during the life of the object.

In practice, we rarely use the `id()` function while programming. Identity checks are most often done with the `is` operator, which compares the object IDs, so our code doesn’t need to call `id()` explicitly. Next, we’ll talk about `is` versus `==`.



For tech reviewer Leonardo Rochael, the most frequent use for `id()` is while debugging, when the `repr()` of two objects look alike, but you need to understand whether two references are aliases or point to separate objects. If the references are in different contexts—such as different stack frames—using the `is` operator may not be viable.

Choosing Between `==` and `is`

The `==` operator compares the values of objects (the data they hold), while `is` compares their identities.

While programming, we often care more about values than object identities, so `==` appears more frequently than `is` in Python code.

However, if you are comparing a variable to a singleton, then it makes sense to use `is`. By far, the most common case is checking whether a variable is bound to `None`. This is the recommended way to do it:

```
x is None
```

And the proper way to write its negation is:

```
x is not None
```

`None` is the most common singleton we test with `is`. Sentinel objects are another example of singletons we test with `is`. Here is one way to create and test a sentinel object:

```

END_OF_DATA = object()
# ... many lines
def traverse(...):
    # ... more lines
    if node is END_OF_DATA:
        return
    # etc.

```

The `is` operator is faster than `==`, because it cannot be overloaded, so Python does not have to find and invoke special methods to evaluate it, and computing is as simple as comparing two integer IDs. In contrast, `a == b` is syntactic sugar for `a.__eq__(b)`. The `__eq__` method inherited from `object` compares object IDs, so it produces the same result as `is`. But most built-in types override `__eq__` with more meaningful implementations that actually take into account the values of the object attributes. Equality may involve a lot of processing—for example, when comparing large collections or deeply nested structures.



Usually we are more interested in object equality than identity. Checking for `None` is the *only* common use case for the `is` operator. Most other uses I see while reviewing code are wrong. If you are not sure, use `==`. It's usually what you want, and also works with `None`—albeit not as fast.

To wrap up this discussion of identity versus equality, we'll see that the famously immutable tuple is not as unchanging as you may expect.

The Relative Immutability of Tuples

Tuples, like most Python collections—lists, dicts, sets, etc.—are containers: they hold references to objects.² If the referenced items are mutable, they may change even if the tuple itself does not. In other words, the immutability of tuples really refers to the physical contents of the tuple data structure (i.e., the references it holds), and does not extend to the referenced objects.

Example 6-5 illustrates the situation in which the value of a tuple changes as a result of changes to a mutable object referenced in it. What can never change in a tuple is the identity of the items it contains.

² In contrast, flat sequences like `str`, `bytes`, and `array.array` don't contain references but directly hold their contents—characters, bytes, and numbers—in contiguous memory.

Example 6-5. t1 and t2 initially compare equal, but changing a mutable item inside tuple t1 makes it different

```
>>> t1 = (1, 2, [30, 40]) ❶
>>> t2 = (1, 2, [30, 40]) ❷
>>> t1 == t2 ❸
True
>>> id(t1[-1]) ❹
4302515784
>>> t1[-1].append(99) ❺
>>> t1
(1, 2, [30, 40, 99])
>>> id(t1[-1]) ❻
4302515784
>>> t1 == t2 ❼
False
```

- ❶ t1 is immutable, but t1[-1] is mutable.
- ❷ Build a tuple t2 whose items are equal to those of t1.
- ❸ Although distinct objects, t1 and t2 compare equal, as expected.
- ❹ Inspect the identity of the list at t1[-1].
- ❺ Modify the t1[-1] list in place.
- ❻ The identity of t1[-1] has not changed, only its value.
- ❼ t1 and t2 are now different.

This relative immutability of tuples is behind the riddle “[A += Assignment Puzzler](#)” on page 54. It’s also the reason why some tuples are unhashable, as we’ve seen in “[What Is Hashable](#)” on page 84.

The distinction between equality and identity has further implications when you need to copy an object. A copy is an equal object with a different ID. But if an object contains other objects, should the copy also duplicate the inner objects, or is it OK to share them? There’s no single answer. Read on for a discussion.

Copies Are Shallow by Default

The easiest way to copy a list (or most built-in mutable collections) is to use the built-in constructor for the type itself. For example:

```
>>> l1 = [3, [55, 44], (7, 8, 9)]
>>> l2 = list(l1) ❶
```

```

>>> l2
[3, [55, 44], (7, 8, 9)]
>>> l2 == l1 ❷
True
>>> l2 is l1 ❸
False

```

- ❶ `list(l1)` creates a copy of `l1`.
- ❷ The copies are equal...
- ❸ ...but refer to two different objects.

For lists and other mutable sequences, the shortcut `l2 = l1[:]` also makes a copy.

However, using the constructor or `[:]` produces a *shallow copy* (i.e., the outermost container is duplicated, but the copy is filled with references to the same items held by the original container). This saves memory and causes no problems if all the items are immutable. But if there are mutable items, this may lead to unpleasant surprises.

In [Example 6-6](#), we create a shallow copy of a list containing another list and a tuple, and then make changes to see how they affect the referenced objects.



If you have a connected computer on hand, I highly recommend watching the interactive animation for [Example 6-6](#) at the [Online Python Tutor](#). As I write this, direct linking to a prepared example at [pythontutor.com](#) is not working reliably, but the tool is awesome, so taking the time to copy and paste the code is worthwhile.

Example 6-6. Making a shallow copy of a list containing another list; copy and paste this code to see it animated at the [Online Python Tutor](#)

```

l1 = [3, [66, 55, 44], (7, 8, 9)]
l2 = list(l1) ❶
l1.append(100) ❷
l1[1].remove(55) ❸
print('l1:', l1)
print('l2:', l2)
l2[1] += [33, 22] ❹
l2[2] += (10, 11) ❺
print('l1:', l1)
print('l2:', l2)

```

- ❶ `l2` is a shallow copy of `l1`. This state is depicted in [Figure 6-3](#).
- ❷ Appending `100` to `l1` has no effect on `l2`.

- ③ Here we remove 55 from the inner list `l1[1]`. This affects `l2` because `l2[1]` is bound to the same list as `l1[1]`.
- ④ For a mutable object like the list referred by `l2[1]`, the operator `+=` changes the list in place. This change is visible at `l1[1]`, which is an alias for `l2[1]`.
- ⑤ `+=` on a tuple creates a new tuple and rebinds the variable `l2[2]` here. This is the same as doing `l2[2] = l2[2] + (10, 11)`. Now the tuples in the last position of `l1` and `l2` are no longer the same object. See [Figure 6-4](#).

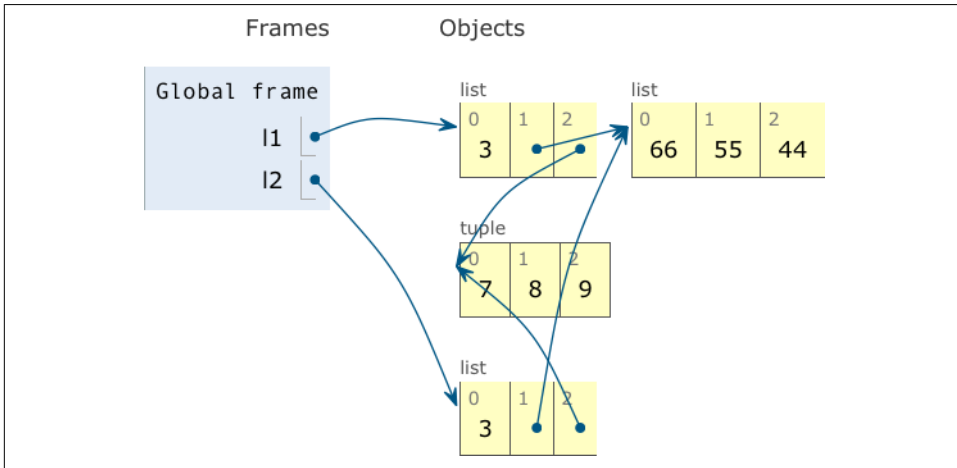


Figure 6-3. Program state immediately after the assignment `l2 = list(l1)` in [Example 6-6](#). `l1` and `l2` refer to distinct lists, but the lists share references to the same inner list object `[66, 55, 44]` and tuple `(7, 8, 9)`. (Diagram generated by the [Online Python Tutor](#).)

The output of [Example 6-6](#) is [Example 6-7](#), and the final state of the objects is depicted in [Figure 6-4](#).

Example 6-7. Output of [Example 6-6](#)

```
l1: [3, [66, 44], (7, 8, 9), 100]
l2: [3, [66, 44], (7, 8, 9)]
l1: [3, [66, 44, 33, 22], (7, 8, 9), 100]
l2: [3, [66, 44, 33, 22], (7, 8, 9, 10, 11)]
```

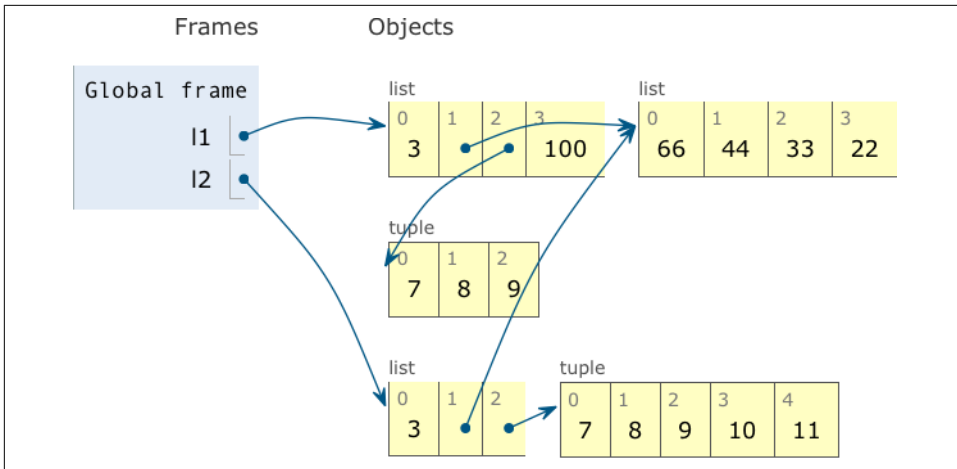


Figure 6-4. Final state of `l1` and `l2`: they still share references to the same list object, now containing `[66, 44, 33, 22]`, but the operation `l2[2] += (10, 11)` created a new tuple with content `(7, 8, 9, 10, 11)`, unrelated to the tuple `(7, 8, 9)` referenced by `l1[2]`. (Diagram generated by the Online Python Tutor.)

It should be clear now that shallow copies are easy to make, but they may or may not be what you want. How to make deep copies is our next topic.

Deep and Shallow Copies of Arbitrary Objects

Working with shallow copies is not always a problem, but sometimes you need to make deep copies (i.e., duplicates that do not share references of embedded objects). The `copy` module provides the `deepcopy` and `copy` functions that return deep and shallow copies of arbitrary objects.

To illustrate the use of `copy()` and `deepcopy()`, [Example 6-8](#) defines a simple class, `Bus`, representing a school bus that is loaded with passengers and then picks up or drops off passengers on its route.

Example 6-8. Bus picks up and drops off passengers

```
class Bus:

    def __init__(self, passengers=None):
        if passengers is None:
            self.passengers = []
        else:
            self.passengers = list(passengers)

    def pick(self, name):
        self.passengers.append(name)
```



```
def drop(self, name):
    self.passengers.remove(name)
```

Now, in the interactive [Example 6-9](#), we will create a bus object (bus1) and two clones—a shallow copy (bus2) and a deep copy (bus3)—to observe what happens as bus1 drops off a student.

Example 6-9. Effects of using copy versus deepcopy

```
>>> import copy
>>> bus1 = Bus(['Alice', 'Bill', 'Claire', 'David'])
>>> bus2 = copy.copy(bus1)
>>> bus3 = copy.deepcopy(bus1)
>>> id(bus1), id(bus2), id(bus3)
(4301498296, 4301499416, 4301499752) ❶
>>> bus1.drop('Bill')
>>> bus2.passengers
['Alice', 'Claire', 'David'] ❷
>>> id(bus1.passengers), id(bus2.passengers), id(bus3.passengers)
(4302658568, 4302658568, 4302657800) ❸
>>> bus3.passengers
['Alice', 'Bill', 'Claire', 'David'] ❹
```

- ❶ Using copy and deepcopy, we create three distinct Bus instances.
- ❷ After bus1 drops 'Bill', he is also missing from bus2.
- ❸ Inspection of the passengers attributes shows that bus1 and bus2 share the same list object, because bus2 is a shallow copy of bus1.
- ❹ bus3 is a deep copy of bus1, so its passengers attribute refers to another list.

Note that making deep copies is not a simple matter in the general case. Objects may have cyclic references that would cause a naïve algorithm to enter an infinite loop. The deepcopy function remembers the objects already copied to handle cyclic references gracefully. This is demonstrated in [Example 6-10](#).

Example 6-10. Cyclic references: b refers to a, and then is appended to a; deepcopy still manages to copy a

```
>>> a = [10, 20]
>>> b = [a, 30]
>>> a.append(b)
>>> a
[10, 20, [[...], 30]]
>>> from copy import deepcopy
```

```
>>> c = deepcopy(a)
>>> c
[10, 20, [[...], 30]]
```

Also, a deep copy may be too deep in some cases. For example, objects may refer to external resources or singletons that should not be copied. You can control the behavior of both `copy` and `deepcopy` by implementing the `__copy__()` and `__deepcopy__()` special methods, as described in the [copy module documentation](#).

The sharing of objects through aliases also explains how parameter passing works in Python, and the problem of using mutable types as parameter defaults. These issues will be covered next.

Function Parameters as References

The only mode of parameter passing in Python is *call by sharing*. That is the same mode used in most object-oriented languages, including JavaScript, Ruby, and Java (this applies to Java reference types; primitive types use call by value). Call by sharing means that each formal parameter of the function gets a copy of each reference in the arguments. In other words, the parameters inside the function become aliases of the actual arguments.

The result of this scheme is that a function may change any mutable object passed as a parameter, but it cannot change the identity of those objects (i.e., it cannot altogether replace an object with another). [Example 6-11](#) shows a simple function using `+=` on one of its parameters. As we pass numbers, lists, and tuples to the function, the actual arguments passed are affected in different ways.

Example 6-11. A function may change any mutable object it receives

```
>>> def f(a, b):
...     a += b
...     return a
...
>>> x = 1
>>> y = 2
>>> f(x, y)
3
>>> x, y ❶
(1, 2)
>>> a = [1, 2]
>>> b = [3, 4]
>>> f(a, b)
[1, 2, 3, 4]
>>> a, b ❷
([1, 2, 3, 4], [3, 4])
>>> t = (10, 20)
```

```
>>> u = (30, 40)
>>> f(t, u) ❸
(10, 20, 30, 40)
>>> t, u
((10, 20), (30, 40))
```

- ❶ The number `x` is unchanged.
- ❷ The list `a` is changed.
- ❸ The tuple `t` is unchanged.

Another issue related to function parameters is the use of mutable values for defaults, as discussed next.

Mutable Types as Parameter Defaults: Bad Idea

Optional parameters with default values are a great feature of Python function definitions, allowing our APIs to evolve while remaining backward compatible. However, you should avoid mutable objects as default values for parameters.

To illustrate this point, in [Example 6-12](#), we take the `Bus` class from [Example 6-8](#) and change its `__init__` method to create `HauntedBus`. Here we tried to be clever, and instead of having a default value of `passengers=None`, we have `passengers=[]`, thus avoiding the `if` in the previous `__init__`. This “cleverness” gets us into trouble.

Example 6-12. A simple class to illustrate the danger of a mutable default

```
class HauntedBus:
    """A bus model haunted by ghost passengers"""

    def __init__(self, passengers=[]): ❶
        self.passengers = passengers ❷

    def pick(self, name):
        self.passengers.append(name) ❸

    def drop(self, name):
        self.passengers.remove(name)
```

- ❶ When the `passengers` argument is not passed, this parameter is bound to the default list object, which is initially empty.
- ❷ This assignment makes `self.passengers` an alias for `passengers`, which is itself an alias for the default list, when no `passengers` argument is given.

- ❸ When the methods `.remove()` and `.append()` are used with `self.passengers`, we are actually mutating the default list, which is an attribute of the function object.

Example 6-13 shows the eerie behavior of the `HauntedBus`.

Example 6-13. Buses haunted by ghost passengers

```
>>> bus1 = HauntedBus(['Alice', 'Bill']) ❶
>>> bus1.passengers
['Alice', 'Bill']
>>> bus1.pick('Charlie')
>>> bus1.drop('Alice')
>>> bus1.passengers ❷
['Bill', 'Charlie']
>>> bus2 = HauntedBus() ❸
>>> bus2.pick('Carrie')
>>> bus2.passengers
['Carrie']
>>> bus3 = HauntedBus() ❹
>>> bus3.passengers ❺
['Carrie']
>>> bus3.pick('Dave')
>>> bus2.passengers ❻
['Carrie', 'Dave']
>>> bus2.passengers is bus3.passengers ❼
True
>>> bus1.passengers ❽
['Bill', 'Charlie']
```

- ❶ bus1 starts with a two-passenger list.
- ❷ So far, so good: no surprises with bus1.
- ❸ bus2 starts empty, so the default empty list is assigned to `self.passengers`.
- ❹ bus3 also starts empty, again the default list is assigned.
- ❺ The default is no longer empty!
- ❻ Now Dave, picked by bus3, appears in bus2.
- ❼ The problem: `bus2.passengers` and `bus3.passengers` refer to the same list.
- ❽ But `bus1.passengers` is a distinct list.

The problem is that `HauntedBus` instances that don't get an initial passenger list end up sharing the same passenger list among themselves.

Such bugs may be subtle. As [Example 6-13](#) demonstrates, when a `HauntedBus` is instantiated with passengers, it works as expected. Strange things happen only when a `HauntedBus` starts empty, because then `self.passengers` becomes an alias for the default value of the `passengers` parameter. The problem is that each default value is evaluated when the function is defined—i.e., usually when the module is loaded—and the default values become attributes of the function object. So if a default value is a mutable object, and you change it, the change will affect every future call of the function.

After running the lines in [Example 6-13](#), you can inspect the `HauntedBus.__init__` object and see the ghost students haunting its `__defaults__` attribute:

```
>>> dir(HauntedBus.__init__) # doctest: +ELLIPSIS
['__annotations__', '__call__', ..., '__defaults__', ...]
>>> HauntedBus.__init__.__defaults__
(['Carrie', 'Dave'],)
```

Finally, we can verify that `bus2.passengers` is an alias bound to the first element of the `HauntedBus.__init__.__defaults__` attribute:

```
>>> HauntedBus.__init__.__defaults__[0] is bus2.passengers
True
```

The issue with mutable defaults explains why `None` is commonly used as the default value for parameters that may receive mutable values. In [Example 6-8](#), `__init__` checks whether the `passengers` argument is `None`. If it is, `self.passengers` is bound to a new empty list. If `passengers` is not `None`, the correct implementation binds a copy of that argument to `self.passengers`. The next section explains why copying the argument is a good practice.

Defensive Programming with Mutable Parameters

When you are coding a function that receives a mutable parameter, you should carefully consider whether the caller expects the argument passed to be changed.

For example, if your function receives a `dict` and needs to modify it while processing it, should this side effect be visible outside of the function or not? Actually it depends on the context. It's really a matter of aligning the expectation of the coder of the function and that of the caller.

The last bus example in this chapter shows how a `TwilightBus` breaks expectations by sharing its passenger list with its clients. Before studying the implementation, see in [Example 6-14](#) how the `TwilightBus` class works from the perspective of a client of the class.

Example 6-14. Passengers disappear when dropped by a TwilightBus

```
>>> basketball_team = ['Sue', 'Tina', 'Maya', 'Diana', 'Pat'] ❶
>>> bus = TwilightBus(basketball_team) ❷
>>> bus.drop('Tina') ❸
>>> bus.drop('Pat')
>>> basketball_team ❹
['Sue', 'Maya', 'Diana']
```

- ❶ basketball_team holds five student names.
- ❷ A TwilightBus is loaded with the team.
- ❸ The bus drops one student, then another.
- ❹ The dropped passengers vanished from the basketball team!

TwilightBus violates the “Principle of least astonishment,” a best practice of interface design.³ It surely is astonishing that when the bus drops a student, their name is removed from the basketball team roster.

Example 6-15 is the implementation TwilightBus and an explanation of the problem.

Example 6-15. A simple class to show the perils of mutating received arguments

```
class TwilightBus:
    """A bus model that makes passengers vanish"""

    def __init__(self, passengers=None):
        if passengers is None:
            self.passengers = [] ❶
        else:
            self.passengers = passengers ❷

    def pick(self, name):
        self.passengers.append(name)

    def drop(self, name):
        self.passengers.remove(name) ❸
```

³ See *Principle of least astonishment* in the English Wikipedia.

- ❶ Here we are careful to create a new empty list when `passengers` is `None`.
- ❷ However, this assignment makes `self.passengers` an alias for `passengers`, which is itself an alias for the actual argument passed to `__init__` (i.e., `basketball_team` in [Example 6-14](#)).
- ❸ When the methods `.remove()` and `.append()` are used with `self.passengers`, we are actually mutating the original list received as an argument to the constructor.

The problem here is that the bus is aliasing the list that is passed to the constructor. Instead, it should keep its own passenger list. The fix is simple: in `__init__`, when the `passengers` parameter is provided, `self.passengers` should be initialized with a copy of it, as we did correctly in [Example 6-8](#):

```
def __init__(self, passengers=None):
    if passengers is None:
        self.passengers = []
    else:
        self.passengers = list(passengers) ❶
```

- ❶ Make a copy of the `passengers` list, or convert it to a list if it's not one.

Now our internal handling of the passenger list will not affect the argument used to initialize the bus. As a bonus, this solution is more flexible: now the argument passed to the `passengers` parameter may be a tuple or any other iterable, like a set or even database results, because the `list` constructor accepts any iterable. As we create our own list to manage, we ensure that it supports the necessary `.remove()` and `.append()` operations we use in the `.pick()` and `.drop()` methods.



Unless a method is explicitly intended to mutate an object received as an argument, you should think twice before aliasing the argument object by simply assigning it to an instance variable in your class. If in doubt, make a copy. Your clients will be happier. Of course, making a copy is not free: there is a cost in CPU and memory. However, an API that causes subtle bugs is usually a bigger problem than one that is a little slower or uses more resources.

Now let's talk about one of the most misunderstood of Python's statements: `del`.

del and Garbage Collection

Objects are never explicitly destroyed; however, when they become unreachable they may be garbage-collected.

—“Data Model” chapter of *The Python Language Reference*

The first strange fact about `del` is that it’s not a function, it’s a statement. We write `del x` and not `del(x)`—although the latter also works, but only because the expressions `x` and `(x)` usually mean the same thing in Python.

The second surprising fact is that `del` deletes references, not objects. Python’s garbage collector may discard an object from memory as an indirect result of `del`, if the deleted variable was the last reference to the object. Rebinding a variable may also cause the number of references to an object to reach zero, causing its destruction.

```
>>> a = [1, 2] ❶
>>> b = a      ❷
>>> del a      ❸
>>> b          ❹
[1, 2]
>>> b = [3]    ❺
```

- ❶ Create object `[1, 2]` and bind `a` to it.
- ❷ Bind `b` to the same `[1, 2]` object.
- ❸ Delete reference `a`.
- ❹ `[1, 2]` was not affected, because `b` still points to it.
- ❺ Rebinding `b` to a different object removes the last remaining reference to `[1, 2]`. Now the garbage collector can discard that object.



There is a `__del__` special method, but it does not cause the disposal of the instance, and should not be called by your code. `__del__` is invoked by the Python interpreter when the instance is about to be destroyed to give it a chance to release external resources. You will seldom need to implement `__del__` in your own code, yet some Python programmers spend time coding it for no good reason. The proper use of `__del__` is rather tricky. See the [__del__ special method documentation](#) in the “Data Model” chapter of *The Python Language Reference*.

In CPython, the primary algorithm for garbage collection is reference counting. Essentially, each object keeps count of how many references point to it. As soon as

that *refcount* reaches zero, the object is immediately destroyed: CPython calls the `__del__` method on the object (if defined) and then frees the memory allocated to the object. In CPython 2.0, a generational garbage collection algorithm was added to detect groups of objects involved in reference cycles—which may be unreachable even with outstanding references to them, when all the mutual references are contained within the group. Other implementations of Python have more sophisticated garbage collectors that do not rely on reference counting, which means the `__del__` method may not be called immediately when there are no more references to the object. See “PyPy, Garbage Collection, and a Deadlock” by A. Jesse Jiryu Davis for discussion of improper and proper use of `__del__`.

To demonstrate the end of an object’s life, [Example 6-16](#) uses `weakref.finalize` to register a callback function to be called when an object is destroyed.

Example 6-16. Watching the end of an object when no more references point to it

```
>>> import weakref
>>> s1 = {1, 2, 3}
>>> s2 = s1
>>> def bye():
...     print('...like tears in the rain.')
...
>>> ender = weakref.finalize(s1, bye)
>>> ender.alive
True
>>> del s1
>>> ender.alive
True
>>> s2 = 'spam'
...like tears in the rain.
>>> ender.alive
False
```

- ❶ `s1` and `s2` are aliases referring to the same set, `{1, 2, 3}`.
- ❷ This function must not be a bound method of the object about to be destroyed or otherwise hold a reference to it.
- ❸ Register the `bye` callback on the object referred by `s1`.
- ❹ The `.alive` attribute is `True` before the `finalize` object is called.
- ❺ As discussed, `del` did not delete the object, just the `s1` reference to it.
- ❻ Rebinding the last reference, `s2`, makes `{1, 2, 3}` unreachable. It is destroyed, the `bye` callback is invoked, and `ender.alive` becomes `False`.

The point of [Example 6-16](#) is to make explicit that `del` does not delete objects, but objects may be deleted as a consequence of being unreachable after `del` is used.

You may be wondering why the `{1, 2, 3}` object was destroyed in [Example 6-16](#). After all, the `s1` reference was passed to the `finalize` function, which must have held on to it in order to monitor the object and invoke the callback. This works because `finalize` holds a *weak reference* to `{1, 2, 3}`. Weak references to an object do not increase its reference count. Therefore, a weak reference does not prevent the target object from being garbage collected. Weak references are useful in caching applications because you don't want the cached objects to be kept alive just because they are referenced by the cache.



Weak references is a very specialized topic. That's why I chose to skip it in this second edition. Instead, I published “[Weak References](#)” on [fluentpython.com](#).

Tricks Python Plays with Immutables



This optional section discusses some Python details that are not really important for *users* of Python, and that may not apply to other Python implementations or even future versions of CPython. Nevertheless, I've seen people stumble upon these corner cases and then start using the `is` operator incorrectly, so I felt they were worth mentioning.

I was surprised to learn that, for a tuple `t`, `t[:]` does not make a copy, but returns a reference to the same object. You also get a reference to the same tuple if you write `tuple(t)`.⁴ [Example 6-17](#) proves it.

Example 6-17. A tuple built from another is actually the same exact tuple

```
>>> t1 = (1, 2, 3)
>>> t2 = tuple(t1)
>>> t2 is t1 ❶
True
>>> t3 = t1[:]
>>> t3 is t1 ❷
True
```

⁴ This is clearly documented. Type `help(tuple)` in the Python console to read: “If the argument is a tuple, the return value is the same object.” I thought I knew everything about tuples before writing this book.

- ❶ `t1` and `t2` are bound to the same object.
- ❷ And so is `t3`.

The same behavior can be observed with instances of `str`, `bytes`, and `frozenset`. Note that a `frozenset` is not a sequence, so `fs[:]` does not work if `fs` is a `frozenset`. But `fs.copy()` has the same effect: it cheats and returns a reference to the same object, and not a copy at all, as [Example 6-18](#) shows.⁵

Example 6-18. String literals may create shared objects

```
>>> t1 = (1, 2, 3)
>>> t3 = (1, 2, 3) ❶
>>> t3 is t1 ❷
False
>>> s1 = 'ABC'
>>> s2 = 'ABC' ❸
>>> s2 is s1 ❹
True
```

- ❶ Creating a new tuple from scratch.
- ❷ `t1` and `t3` are equal, but not the same object.
- ❸ Creating a second `str` from scratch.
- ❹ Surprise: `a` and `b` refer to the same `str`!

The sharing of string literals is an optimization technique called *interning*. CPython uses a similar technique with small integers to avoid unnecessary duplication of numbers that appear frequently in programs like 0, 1, -1, etc. Note that CPython does not intern all strings or integers, and the criteria it uses to do so is an undocumented implementation detail.



Never depend on `str` or `int` interning! Always use `==` instead of `is` to compare strings or integers for equality. Interning is an optimization for internal use of the Python interpreter.

⁵ The harmless lie of having the `copy` method not copying anything is justified by interface compatibility: it makes `frozenset` more compatible with `set`. Anyway, it makes no difference to the end user whether two identical immutable objects are the same or are copies.

The tricks discussed in this section, including the behavior of `frozenset.copy()`, are harmless “lies” that save memory and make the interpreter faster. Do not worry about them, they should not give you any trouble because they only apply to immutable types. Probably the best use of these bits of trivia is to win bets with fellow Pythonistas.⁶

Chapter Summary

Every Python object has an identity, a type, and a value. Only the value of an object may change over time.⁷

If two variables refer to immutable objects that have equal values (`a == b` is `True`), in practice it rarely matters if they refer to copies or are aliases referring to the same object, because the value of an immutable object does not change, with one exception. The exception being immutable collections such as tuples: if an immutable collection holds references to mutable items, then its value may actually change when the value of a mutable item changes. In practice, this scenario is not so common. What never changes in an immutable collection are the identities of the objects within. The `frozenset` class does not suffer from this problem because it can only hold hashable elements, and the value of hashable objects cannot ever change, by definition.

The fact that variables hold references has many practical consequences in Python programming:

- Simple assignment does not create copies.
- Augmented assignment with `+=` or `*=` creates new objects if the lefthand variable is bound to an immutable object, but may modify a mutable object in place.
- Assigning a new value to an existing variable does not change the object previously bound to it. This is called a rebinding: the variable is now bound to a different object. If that variable was the last reference to the previous object, that object will be garbage collected.

⁶ A terrible use for this information would be to ask about it when interviewing candidates or authoring questions for “certification” exams. There are countless more important and useful facts to check for Python knowledge.

⁷ Actually the type of an object may be changed by merely assigning a different class to its `__class__` attribute, but that is pure evil and I regret writing this footnote.

- Function parameters are passed as aliases, which means the function may change any mutable object received as an argument. There is no way to prevent this, except making local copies or using immutable objects (e.g., passing a tuple instead of a list).
- Using mutable objects as default values for function parameters is dangerous because if the parameters are changed in place, then the default is changed, affecting every future call that relies on the default.

In CPython, objects are discarded as soon as the number of references to them reaches zero. They may also be discarded if they form groups with cyclic references but not outside references.

In some situations, it may be useful to hold a reference to an object that will not—by itself—keep an object alive. One example is a class that wants to keep track of all its current instances. This can be done with weak references, a low-level mechanism underlying the more useful collections `WeakValueDictionary`, `WeakKeyDictionary`, `WeakSet`, and the `finalize` function from the `weakref` module. For more on this, please see “[Weak References](#)” at fluentpython.com.

Further Reading

The “[Data Model](#)” chapter of *The Python Language Reference* starts with a clear explanation of object identities and values.

Wesley Chun, author of the *Core Python* series of books, presented [Understanding Python’s Memory Model, Mutability, and Methods](#) at EuroPython 2011, covering not only the theme of this chapter but also the use of special methods.

Doug Hellmann wrote the posts “[copy – Duplicate Objects](#)” and “[weakref—Garbage-Collectable References to Objects](#)” covering some of the topics we just discussed.

More information on the CPython generational garbage collector can be found in the [gc module documentation](#), which starts with the sentence “This module provides an interface to the optional garbage collector.” The “optional” qualifier here may be surprising, but the “[Data Model](#)” chapter also states:

An implementation is allowed to postpone garbage collection or omit it altogether—it is a matter of implementation quality how garbage collection is implemented, as long as no objects are collected that are still reachable.

PEP 585 – Type Hinting Generics In Standard Collections

Final (Accepted for Python 3.9+)

Łukasz Langa

PEP 585 proposes that built-in collection types (like list, dict, tuple, etc.) should support generic type annotations directly, eliminating the need for importing the corresponding classes from typing (e.g. List, Dict, Tuple).

Before Python 3.9, you had to use typing.List or typing.Dict for type annotations:

```
from typing import List, Dict
```

```
def foo(names: List[str], ages: Dict[str, int]):
```

```
    ...
```

This was unintuitive since list, dict, etc. are already built-in types. PEP 585 allows you to write:

```
def foo(names: list[str], ages: dict[str, int]):
```

```
    ...
```

This improves clarity and reduces the need to import from typing.

Key Changes:

- Built-in collections become generic.
- You can specify element types directly in square brackets.
- Supports nested and advanced types: list[tuple[int, str]], etc.
- typing equivalents (List, Dict, etc.) remain for backward compatibility, but are deprecated in favor of the new syntax.

Benefits:

- Cleaner syntax.
- Fewer imports from typing.
- Consistency with other generic types (like collections.abc and user-defined generics).
- Easier learning curve for new users.

Limitations:

- Only works in Python 3.9 and newer.
- Some specialized types (like Any, Optional, Union) still come from typing.
- Not all third-party tools may have immediate support (though most are now compatible).

Example:

Python 3.8

```
from typing import List
```

```
def squares(nums: List[int]) -> List[int]:
```

```
    return [n * n for n in nums]
```

Python 3.9+

```
def squares(nums: list[int]) -> list[int]:
```

```
    return [n * n for n in nums]
```

Python Generics in Practice**Introduction**

Type hints have transformed Python into a more robust, maintainable language. They enable static checking, better IDE support, and clearer code. But until Python 3.9, specifying the types of collections often felt cumbersome:

```
from typing import List
```

```
def process(items: List[str]):
```

```
    ...
```

Why import List when list is already a built-in? PEP 585 solved this by making built-in collections generic. Now you can write:

```
def process(items: list[str]):
```

```
    ...
```

Cleaner, simpler, and more Pythonic.

Advanced Use: User-Defined Generics

PEP 585 also aligns with how you can define your own generics using `typing.Generic`.

```
from typing import TypeVar, Generic
```

```
T = TypeVar('T')
```

```
class Box(Generic[T]):  
    def __init__(self, value: T):  
        self.value = value  
  
    def get(self) -> T:  
        return self.value
```

Usage:

```
box: Box[int] = Box(42)  
print(box.get()) # 42
```

You can also use built-in types in your own generics:

```
from collections.abc import Iterable
```

```
def total(values: Iterable[int]) -> int:  
    return sum(values)
```


Mini-Exercise: Design a Deque Class with Generics

Problem Statement & Requirements

Background

A **Deque** (double-ended queue) is a data structure that allows insertion and removal of elements from both the front and back.

You are to design a **generic Deque class** in a programming language of your choice (e.g., Java, C++, Python with type hints). Use generics to allow the Deque to hold any type of object.

Requirements

Your Deque class should support the following operations:

addFront(item: T) – Insert item at the front.
addBack(item: T) – Insert item at the back.
removeFront(): T – Remove and return item from the front.
removeBack(): T – Remove and return item from the back.
peekFront(): T – Return (without removing) item at the front.
peekBack(): T – Return (without removing) item at the back.
isEmpty(): boolean – Return true if the deque is empty.
size(): int – Return the number of elements.

Constraints

- Use **generics** to make your class type-safe for any data type.
- Internally, you may use a dynamic array or a linked list.
- Do **not** use built-in language library deque/queue classes.

Example Usage

```
Deque<Integer> deque = new Deque<>();  
deque.addFront(1);  
deque.addBack(2);  
System.out.println(deque.removeFront()); // 1  
System.out.println(deque.removeBack()); // 2
```

Tasks

1. Define the class header with generics.
2. Choose an internal data structure.

3. Implement all required methods.
4. Write a short main or test function to show your class works.

Suggested Skeleton and Hints

Skeleton Example (Java-style Pseudocode)

```
public class Deque<T> {  
    // Suggested internal storage  
    private Node<T> head;  
    private Node<T> tail;  
    private int size;  
  
    private static class Node<T> {  
        T data;  
        Node<T> next;  
        Node<T> prev;  
    }  
  
    public Deque() {  
        head = null;  
        tail = null;  
        size = 0;  
    }  
  
    public void addFront(T item) { /* ... */ }  
    public void addBack(T item) { /* ... */ }  
    public T removeFront() { /* ... */ }  
    public T removeBack() { /* ... */ }  
    public T peekFront() { /* ... */ }  
    public T peekBack() { /* ... */ }  
    public boolean isEmpty() { return size == 0; }  
    public int size() { return size; }  
}
```

```
}
```

Stretch Challenge

Add `clear()` to remove all elements.

Add an iterator to allow for loops over the deque.

Optional: Python with Generics (Type Hints)

```
from typing import TypeVar, Generic, Optional
```

```
T = TypeVar('T')
```

```
class Node(Generic[T]):
```

```
    def __init__(self, data: T):
```

```
        self.data = data
```

```
        self.next: Optional['Node[T]'] = None
```

```
        self.prev: Optional['Node[T]'] = None
```

```
class Deque(Generic[T]):
```

```
    def __init__(self):
```

```
        self.head: Optional[Node[T]] = None
```

```
        self.tail: Optional[Node[T]] = None
```

```
        self._size = 0
```

```
    def add_front(self, item: T) -> None: pass
```

```
    def add_back(self, item: T) -> None: pass
```

```
    def remove_front(self) -> T: pass
```

```
    def remove_back(self) -> T: pass
```

```
    def peek_front(self) -> T: pass
```

```
    def peek_back(self) -> T: pass
```

```
    def is_empty(self) -> bool: return self._size == 0
```

```
    def size(self) -> int: return self._size
```